



الجمهورية العربية السورية

جامعة دمشق

كلية الهندسة المعلوماتية

محرك المعالجة عالي الأداء لنظام التجارة الإلكترونية

تقرير إنجاز مشروع مادة البرمجة المتوازية

بيسان محمد ديب

نديم زهير التكلة

علا محمد البيطار

جنى طارق دسوقي

بإشراف: أ. حذيفة عقيل

حماية البيانات المشتركة من التضارب (Concurrent Access & Data Integrity):

1. إنشاء Throttle Interceptor:
تعتمد على مبدأ إنتاج مفتاح key مشفر لكل request حسب معلوماته مثل نوعه والبيانات المرسله، ثم تخزينه في map، عند وصول طلب جديد يتم مقارنة المفاتيح مع بعضها وفي حال التطابق لا ينفذ الطلب الجديد، مع حذف المفتاح بعد مدة معينة time-to-live.
الإيجابيات: منع spam بنفس payload.
السلبيات: التخزين يتم في instance واحد محلي، ttl ليس مرتبط بوقت التنفيذ.
2. القفل المتشائم Pessimistic Locking:
عند قراءة المنتج لتحديث الكمية، تقوم بقله مباشرة بحيث لا يستطيع أي طلب آخر تعديله حتى انتهاء العملية.
الإيجابيات: يمنع التضارب تماماً، مناسب للعمليات الحساسة جداً (مثل الدفع)، بسيط التنفيذ.
السلبيات: يقلل الأداء، ممكن يسبب deadlocks، غير مناسب عند الضغط العالي لذلك يستحسن استخدامه عند التحديثات النادرة ولكن المهمة جداً.
3. القفل المتفائل Optimistic Locking:
لا تقوم بقفل البيانات، بل تفترض أن التضارب نادر، وتتحقق عند التحديث. تقوم بإضافة عمود يعبر عن النسخة، إذا تغيرت يعني شخص آخر عدل البيانات.
الإيجابيات: أداء عالي، مناسب للأنظمة الكبيرة.
السلبيات: يحتاج إعادة محاولة، يفشل عند التضارب العالي، أعقد في التعامل.
4. العمليات الذرية Atomic Updates:
بدل القراءة ثم التعديل برمجياً ثم حفظ النتيجة، يتم التحقق والتعديل مباشرة في قاعدة البيانات.
الإيجابيات: سريع جداً، يقلل أخطاء بشرية.
السلبيات: محدود في المنطق المعقد.
5. العزل للمعاملات Isolation Levels:
درجات حماية تمنع التداخل بين العمليات، لكن كلما زادت الحماية، انخفض الأداء. يوجد 4 أنواع من الأضعف للأقوى:
 - READ UNCOMMITTED: أي معاملة يمكنها قراءة بيانات غير مثبتة قد تلغى لاحقاً أي مشكلة Dirty Read.
 - READ COMMITTED: المعاملات لا تقرأ إلا البيانات المثبتة. يواجه مشكلة Non-Repeatable Read أي نفس الاستعلام يعطي نتيجة مختلفة داخل نفس المعاملة.
 - REPEATABLE READ: أي قيمة تقرأها داخل المعاملة تبقى ثابتة حتى النهاية. يواجه مشكلة Phantom Read أي قد تظهر أو تختفي صفوف أثناء نفس المعاملة.
 - SERIALIZABLE: قاعدة البيانات تتصرف كأن جميع العمليات تنفذ واحدة بعد الأخرى حتى لو فعلياً تعمل بالتوازي. يحل جميع المشاكل السابقة ولكن يواجه مشكلة بالأداء لأن النظام يصبح كأنه تسلسلي مما يسبب بطء.
6. القفل الموزع Distributed Locking:
في حالة وجود أكثر من سيرفر، نستخدم Redis lock مما يسمح لسيرفر واحد فقط بتعديل المنتج.
الإيجابيات: مناسب للأنظمة الموزعة، سريع جداً، يحل مشكلة race conditions بين السيرفرات.
السلبيات: معقد، احتمال فقدان القفل عند سقوط السيرفر.
7. تنظيم طوابير Event Queue:
بدل التحديث المباشر، تُرسل العملية لطاوير ويتم معالجته عن طريق عامل واحد بشكل تسلسلي.
الإيجابيات: يقلل التزامن، فصل بين الطلب والمعالجة وبالتالي قابل للتوسع.
السلبيات: زيادة زمن الاستجابة.

الحل المعتمد:

انطلاقاً من عدم وجود تقنية واحدة قادرة على حل جميع مشاكل التزامن بكفاءة عالية، تم تصميم آلية حماية متعددة الطبقات تعتمد على دمج تقنيتين تعملان معاً لضمان عدم حدوث أي نوع من أنواع التضارب:

1. Redis lock: على مستوى السيرفرات، يمنع تنفيذ نفس العملية بالتوازي بين عدة سيرفرات، يمنع الطلبات المتكررة وحالة .multiserver race conditions.
2. Atomic Update: يحل مشكلة race conditions على مستوى قاعدة البيانات

الاختبار:

تم اختبار إرسال ألف مستخدم طلب شراء نفس المنتج الذي يوجد منه 100 عنصر في المخزن فقط.

- قبل: نلاحظ المخزون أصبح سالب وتم حدوث oversell، نجد في ملف summary نسبة ال error صفر أي تم تنفيذ جميع الطلبات رغم نفاذ المخزون.

<	>	users	×	orders_items	×	products	×
id	name		description	price	stock	created_at	updated_at
1	Recycled Bronze Car		Introducing...	705.25	-900	2026-05-10 12:19:07.19	2026-05-10 15:59:58.849

- بعد: المخزون توقف عند نفاذ الكمية وتم رفض بقية الطلبات

<	>	products						
id		name	description	price	stock	created_at	updated_at	
1		Recycl...	Introduc...	705.25	0	2026-05-06...	2026-05-10 14:12:44.414	

كما نجد في ملف summary ارتفاع نسبة error فنستنتج ان السيرفر لم يسمح بتلبية جميع الطلبات

إدارة المواد الحاسوبية (Resource Management & Capacity Control):

1. تطبيق Rate Limiting:
للتحكم في عدد العمليات المتوازية التي ينفذها النظام وتقييد عدد الطلبات التي يمكن لمستخدم واحد أو عنوان IP واحد إرسالها خلال نافذة زمنية معينة. مما يحمي الذاكرة والمعالج من الاستهلاك المفرط.
الإيجابيات: حماية فورية من هجمات DoS، ومنع العمليات المتكررة غير الضرورية وبالتالي حماية الـ Event Loop من الازدحام الشديد الذي يؤدي لزيادة زمن الاستجابة.
السلبات: قد تمنع مستخدمين حقيقيين من الوصول إذا كانت الحدود ضيقة جداً، ولا تحل مشكلة العمليات الثقيلة داخلياً.
2. تجميع الخيوط (Thread Pools / Worker Threads):
بشكل افتراضي تستخدم node.js مكتبة libuv والتي تمتلك thread pool داخليا للتعامل مع مهام الإدخال والإخراج ولكن لا يمكن استخدامه مباشرة. يمكن بناؤه يدوياً لمعالجة العمليات الثقيلة على cpu.
الإيجابيات: استغلال كامل لقدرات المعالج، منع تجمد المسار الرئيسي للنظام Event Loop، منع الانهيار عند الضغط، إذا تعطل خيط لأي سبب لا يتأثر السيرفر الرئيسي ويستمر في تلقي الطلبات.
السلبات: استهلاك عالي للذاكرة لكل Worker، وتعقيد في تبادل البيانات بين الخيوط.
3. تنظيم طوابير (Request Queuing & Backpressure):
عندما يتجاوز عدد الطلبات قدرة النظام على المعالجة اللحظية، بدلاً من رفضها أو محاولة معالجتها دفعة واحدة (الامر الذي يسبب الانهيار)، نضعها في طابور الانتظار ويتم معالجتها واحدة تلو الأخرى حسب الموارد المتاحة. بالإضافة الى التحقق من عدد الطلبات في الطابور ورفض أي طلب عند الوصول لحد معين (مثلاً 100 طلب ضمن الانتظار).
الإيجابيات: ضمان معالجة كل الطلبات حتى لو تأخرت قليلاً، تحسين استقرار النظام تحت الضغط المفاجئ.
السلبات: زيادة زمن الاستجابة، تتطلب وجود سيرفر خارجي Redis.
4. تحديد موارد الحاويات (Docker):
هذا الحل يتم على مستوى البنية التحتية Infrastructure، بدلاً من ترك التطبيق يستهلك كل موارد السيرفر، نقوم بوضعه داخل حاوية ونحدد سقفاً لا يمكن تجاوزه.
الإيجابيات: عزل كامل للموارد
السلبات: إذا وصل للحد الأقصى للذاكرة سيقوم النظام بقتل العملية OOM Kill مما يتطلب وجود آلية لإعادة التشغيل تلقائياً

الحل المعتمد:

- قمنا ببناء نظام يهدف الى تحقيق توازن بين تنفيذ عدد كافي من العمليات بالتوازي ومنع استهلاك الموارد بشكل مفرط. لتحقيق ذلك:
1. تم تحديد عدد العمليات المتزامنة باستخدام خاصية Concurrency في الطوابير، وفصل الطوابير حسب نوعها.
 2. تم تطبيق Thread pool كخدمة مستقلة داخل النظام باستخدام worker threads.
 3. التحكم في معدل التنفيذ Rate limiting الامر الذي يمنع من الارتفاع المفاجئ في الحمل والتحكم في تدفق الطلبات.
 4. تم ضبط عدد الاتصالات المسموح بها لقاعدة البيانات باستخدام اعدادات Prisma مما يحمي قاعدة البيانات ويضمن الاستقرار.
 5. تطبيق مبدأ Backpressure مما يمنع تراكم الطلبات بشكل غير مُسيطر عليه.

الاختبار:

تم اختبار إرسال ألف مستخدم.

• قبل

Name	Status	81% CPU	74% Memory	1% Disk	0% Network
OpenJDK Platform binary		71.3%	1,546.1 MB	0.4 MB/s	0 Mbps
Thread Group1.jmx (...)					
Visual Studio Code (16)		0.1%	1,293.5 MB	0.1 MB/s	0 Mbps

• بعد

Name	Status	43% CPU	82% Memory	0% Disk	0% Network
OpenJDK Platform binary		0.1%	1,372.1 MB	0 MB/s	0 Mbps
Thread Group1.jmx (...)					
Visual Studio Code		0%	324.6 MB	0.1 MB/s	0 Mbps

تم اختبار إرسال 10 الاف مستخدم.

• بعد

Name	Status	43% CPU	82% Memory	1% Disk	0% Network
OpenJDK Platform binary		0.1%	1,418.8 MB	0.1 MB/s	0 Mbps
Thread Group1.jmx (...)					
Node.js JavaScript Runtime		0%	295.9 MB	0 MB/s	0 Mbps
Visual Studio Code		0%	278.5 MB	0 MB/s	0 Mbps

كما نجد في ملف summary ارتفاع نسبة error، نستنتج ان السيرفر لم يسمح لجميع الطلبات بالدخول معاً واستهلاك موارد المعالج والذاكرة

Response Body	Response headers
{"statusCode":429,"message":"ThrottlerException: Too Many Requests"}	

Prometheus باستخدام مكتبة احصائيات

● قبل

```
# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total 60.37500000000001

# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total 2.6250000000000004

# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 62.99999999999999

# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1778924065

# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 244887552
```

● بعد

```
# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total 30.686999999999998

# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total 2.719

# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total 33.405999999999985

# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1778924219

# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes 253562880
```

نلاحظ انخفاض استهلاك المعالج بنسبة النصف تقريبا

المعالجة غير المتزامنة (Asynchronous Queues):

في البداية، Node.js يعتمد على:

- Event Loop وهي الحلقة التي تدير تنفيذ المهام
- Call stack لتنفيذ العمليات المتزامنة
- Queues للمهام الغير متزامنة
- Libuv مكتبة داخلية تدير I/O و thread pool

1. استخدام Callbacks:

دالة يتم تمريرها كوسيط ليتم تنفيذها لاحقا عند انتهاء العملية. لنفترض لدينا عملية قراءة ملف، يتم تسليمها لمكتبة Libuv التي ستنفذها في الخلفية thread pool ثم عند الانتهاء يتم وضع callback في الطابور لينفذها ال event loop. الإيجابيات: بسيطة وسريعة السلبيات: صعوبة إدارة الأخطاء، callback hell وهي تداخل الدوال، لا يمكن التحكم بسهولة في التسلسل.

2. استخدام Promises:

كائن يمثل نتيجة عملية غير متزامنة مستقبلية (اما نجاح او فشل)، حيث يملك الدالتين callbacks للتعامل مع النجاح او الفشل، عند اكتمال العملية يتم وضعها في الطابور ليتم تنفيذها حيث ستنفذ قبل أي callback تقليدي اخر. الإيجابيات: يدعم chaining، متوافق مع async/await. السلبيات: chaining قد يصبح معقد، صعوبة أحيانا في تتبع الأخطاء.

3. استخدام Async/Await:

Syntax مبسط فوق promises يجعل الكود يبدو متزامنا، حيث await يوقف تنفيذ الدالة فقط وليس event loop وعند اكتمال ال promise يتم استكمال التنفيذ من نفس النقطة. الإيجابيات: معالجة أخطاء باستخدام try/catch. السلبيات: يعتمد بالكامل على promises.

4. استخدام Worker Threads:

تنفيذ الكود في threads منفصلة. يتم انشاء خيط جديد ويكون لكل منهم event loop وذاكرة خاصة به والتواصل يتم عبر message passing. الإيجابيات: توازي حقيقي، مناسب للعمليات الثقيلة، لا يوقف event loop. السلبيات: تعقيد في التواصل، استهلاك ذاكرة أكبر، تتبع أخطاء أصعب.

5. استخدام Background Jobs (Queues):

تنفيذ مهام خارج دورة حياة ال request. يتم وضع job في الطابور ثم عامل منفصل يعالجها ويمكن إعادة المحاولة عند الفشل. الإيجابيات: فصل المهام، قابلية التوسع، الموثوقية. السلبيات: زيادة زمن الاستجابة، تتطلب وجود سيرفر خارجي مثل Redis.

الحل المعتمد:

تم تصميم عملية الشراء بحيث تكون غير متزامنة وتعتمد على نظام طوابير باستخدام BullMQ، مما يسمح بتنفيذ العمليات الثقيلة في الخلفية دون التأثير على استجابة النظام للمستخدم. بدل تنفيذ كل خطوات الشراء مباشرة عند الطلب، يتم إرسال المهمة إلى طابور ليتم معالجتها لاحقا بواسطة Worker مخصص. لدينا ثلاث أنواع من الطوابير: dlq - email - checkout

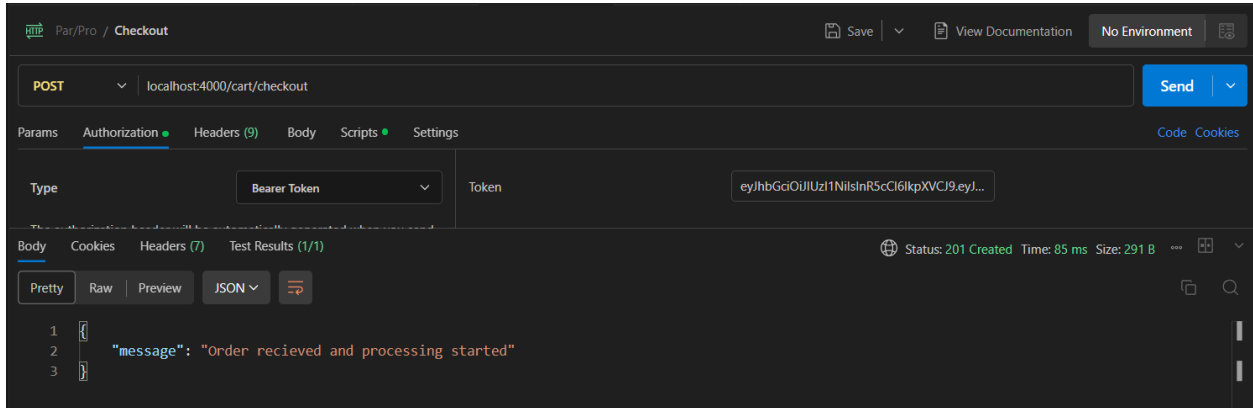
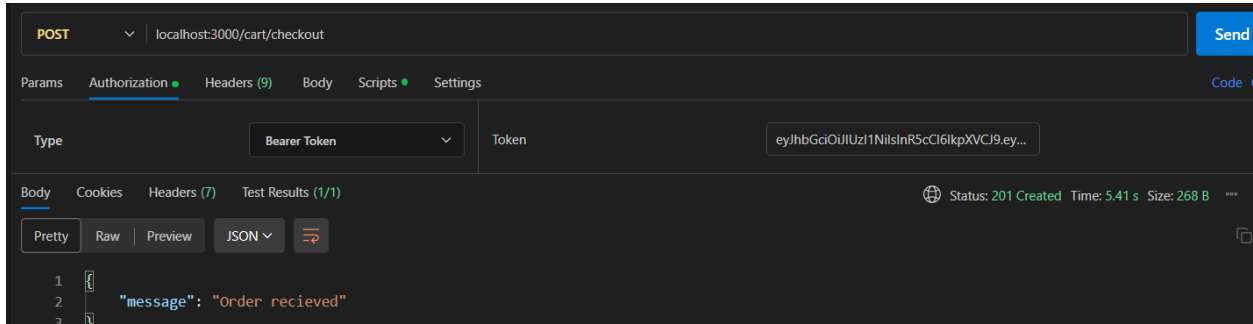
تفاصيل تدفق عملية الشراء:

1. عند قيام المستخدم بإرسال الطلب يتم التأكد بداية من شرطين أساسيين لاستكمال عملية البيع (وجود سلة له ووجود أغراض فيها). عند تحقق الشرطين يتم إرسال عملية الشراء لطابور checkout ليتم معالجتها لاحقاً مع إعلام المستخدم بوصول طلبه.
2. في طابور checkout يقوم العامل ب جلب تفاصيل السلة والتأكد من توفر المخزن وصحة البيانات وحساب المبلغ النهائي، ثم إنشاء طلب جديد في قاعدة البيانات. بعد ذلك تتم عملية الدفع وتحديث حالة المخزون وتفرغ السلة.
3. يتم إضافة عملية إرسال إيميل المستخدم لطابور email ومعالجتها عن طريق عامل منفصل.
4. كل عملية في حالة فشلها يتم إعادة محاولة تنفيذها عدد محدد (3)
5. في حال فشلها بعد كل المحاولات يتم إضافتها لطابور dlq الذي يخزن سبب المشكلة في قاعدة البيانات في جدول loggings ليتم معالجته.

الاختبار:

تم اختبار إرسال مستخدم

- قبل: استغرقت عملية الشراء 5.41 ثانية
- بعد: استغرقت عملية الشراء 85 ميلي ثانية



تم اختبار إرسال ألف مستخدم

- قبل: استغرقت عملية الشراء 641 ميلي ثانية
- بعد: استغرقت عملية الشراء 248 ميلي ثانية

Label ↑	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput
Add to cart	1000	29	2	331	57.58	0.00%	64.3/sec
Checkout	1000	84	8	641	144.61	0.00%	65.6/sec

Label	# Samples	Average	Min	Max ↑	Std. Dev.	Error %	Throughput
Debug Sam...	1000	0	0	1	0.16	0.00%	69.5/sec
Checkout	1000	18	2	248	47.29	76.50%	68.4/sec

تم اختبار إرسال 10 الاف مستخدم

- قبل: استغرقت عملية الشراء 22808 ميلي ثانية
- بعد: استغرقت عملية الشراء 2935 ميلي ثانية

Label	# Samples	Average	Min	Max ↑	Std. Dev.	Throughput
Debug Sam...	20000	0	0	182	3.29	4.4/sec
Add to cart	20000	474	0	6573	880.65	4.4/sec
Checkout	20000	1199	0	22808	3579.72	4.4/sec

Label	# Samples	Average	Min	Max ↑	Std. Dev.	Throughput
Checkout	9685	225	0	2935	279.67	1508.6/sec

نلاحظ الفرق الكبير في زمن الاستجابة قبل تطبيق النظام وبعده

معالجة البيانات الضخمة على دفعات (Batch Processing):

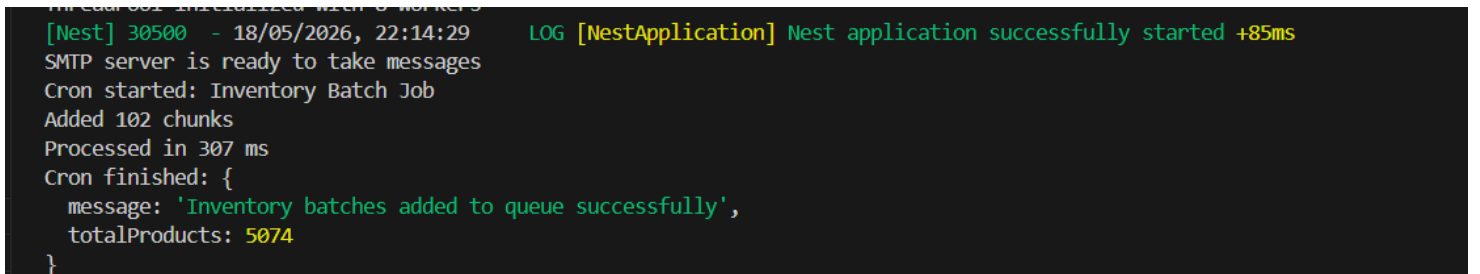
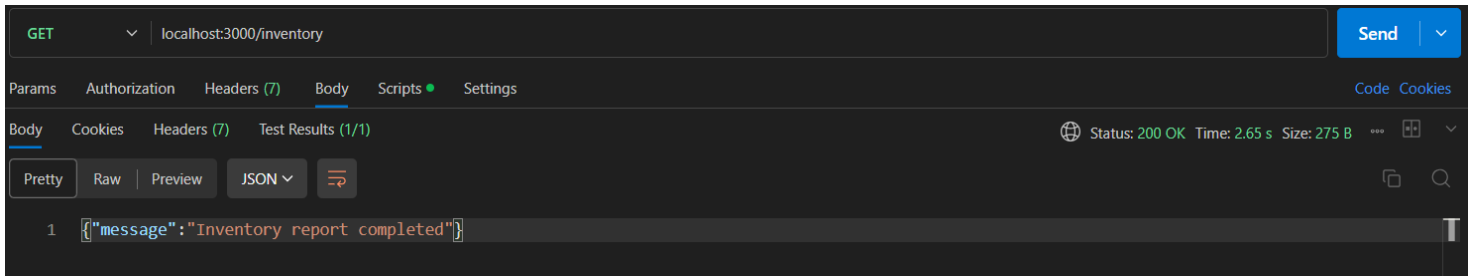
تم تصميم نظام لمعالجة وجرد البيانات الضخمة للمبيعات والمخزون اليومي بكفاءة عالية. يعتمد على استراتيجية المعالجة على دفعات وطوابع الانتظار، حيث يقوم بتقسيم قوائم المنتجات الكبيرة الى مجموعات صغيرة chunks ومعالجتها في الخلفية بالتوازي في طابور مخصص لها، مما يضمن الاستقرار والحماية من الانهيار او البطء اثناء التعامل مع كميات ضخمة. تم استخدام Cron Jobs لتنفيذ الجرد اليومي بشكل تلقائي، يقوم النظام أولاً بمحاولة الحصول على Distributed Lock باستخدام Redis Redlock، الهدف من هذا القفل هو منع تشغيل نفس المهمة أكثر من مرة في حال وجود عدة Services تعمل بالتوازي. أي إذا كان هناك instance آخر قد بدأ المهمة بالفعل فلن يتم تشغيلها مرة ثانية مما يمنع تكرار المعالجة وتضارب البيانات، وبعد انتهاء المهمة يتم تحرير القفل للسماح بالتشغيل التالي بشكل طبيعي.

تفاصيل النظام:

1. يتم تشغيل النظام تلقائياً عن طريق وظيفة مجدولة (Cron job) تعمل كل يوم عند الساعة 12 ليلاً.
2. يتم جلب جميع المنتجات من قاعدة البيانات باستعلام واحد، عوضاً عن تنفيذ استعلام منفصل لكل منتج داخل الدفعة فيؤدي الى عدد كبير من الاستعلامات وضغط على قاعدة البيانات (Query N+1).
3. يتم تقسيم المنتجات الى دفعات صغيرة chunks ثابتة الحجم (50 منتج لكل دفعة).
4. يتم ارسال هذه الدفعات الى طابور كمهام خلفية مع تطبيق توازي بمعدل (5 وظائف) مما يزيد من سرعة المعالجة دون استنزاف الموارد.
5. يتم تحليل بيانات كل منتج داخل الدفعة لحساب كمية المبيعات (المخزون الحالي والمتوقع) وحفظ النتائج في قاعدة البيانات.
6. تم إضافة تنظيف تلقائي للوظيفة بعد اكتمالها او فشلها لمنع تراكم البيانات وضمان استقرار النظام على المدى الطويل.

الاختبار:

- قبل: استغرقت العملية 2.65 ثانية
- بعد: استغرقت حوالي 300 ميلي ثانية



توزيع الأحمال (Load Distribution):

ان النظام لدينا لا يعتمد على خادم واحد فقط لمعالجة الطلبات بل يتم تشغيل أكثر من نسخة من التطبيق داخل حاويات منفصلة باستخدام Docker كل نسخة من التطبيق تعمل بشكل مستقل لكنها تشترك في نفس قاعدة البيانات والخدمات الخلفية بدلا من توجيه المستخدمين مباشرة الى هذه النسخ، استخدمنا خادم وسيط هو NGINX ليقوم بدور موزع الأحمال Load Balancer حيث يتم استقبال جميع الطلبات القادمة من المستخدمين ويقوم بتوزيعها على أكثر من نسخة من التطبيق.

تفاصيل النظام:

1. تم تشغيل عدة نسخ باستخدام docker عبر command `docker compose up --build --scale app=3`
2. كل container يمثل خادما مستقلا
3. بعد ذلك تم اعداد Nginx ليعمل ك Reverse Proxy يقوم باستقبال الطلبات واعادة توجيهها الى نسخ التطبيق
4. الية توزيع الاحمال داخل اعدادات NGINX:
 - تم تعريف مجموعة الخوادم upstream servers تمثل النسخ المختلفة من التطبيق
 - وتم استخدام استراتيجية توزيع تعتمد على Least Connections بحيث يتم ارسال كل طلب الى الخادم الاقل ضغطا

ومن الاختبارات المتكررة قد لاحظنا ان الطلبات يتم توزيعها بين الحاويات كما ان جميع النسخ تعمل على نفس المنطق البرمجي ونفس قاعدة البيانات مما يضمن اتساق البيانات رغم تعدد الخوادم.

```
app-1 | Registered
nest_nginx | 172.20.0.1 - - [18/May/2026:18:42:12 +0000] "POST /auth/register HTTP/1.1" 201 369 "-" "PostmanRuntime/7.39.1" "-"

app-3 | Registered View Config Enable
app-2 | Registered - - [18/May/2026:18:42:
nest_nginx | 172.20.0.1 - - [18/May/2026:18:42:17 +0000] "POST /auth/register HTTP/1.1" 201 369 "-" "PostmanRuntime/7.39.1" "-"
nest_nginx | 172.20.0.1 - - [18/May/2026:18:42:18 +0000] "POST /auth/register HTTP/1.1" 201 369 "-" "PostmanRuntime/7.39.1" "-"
app-1 | Registered

app-3 | Registered
nest_nginx | 172.20.0.1 - - [18/May/2026:18:42:18 +0000] "POST /auth/register HTTP/1.1" 201 369 "-" "PostmanRuntime/7.39.1" "-"
nest_nginx | 172.20.0.1 - - [18/May/2026:18:42:18 +0000] "POST /auth/register HTTP/1.1" 201 369 "-" "PostmanRuntime/7.39.1" "-"
app-2 | Registered
app-1 | Registered
nest_nginx | 172.20.0.1 - - [18/May/2026:18:42:19 +0000] "POST /auth/register HTTP/1.1" 201 369 "-" "PostmanRuntime/7.39.1" "-"
nest_nginx | 172.20.0.1 - - [18/May/2026:18:42:19 +0000] "POST /auth/register HTTP/1.1" 201 369 "-" "PostmanRuntime/7.39.1" "-"
app-3 | Registered
app-2 | Registered
nest_nginx | 172.20.0.1 - - [18/May/2026:18:42:20 +0000] "POST /auth/register HTTP/1.1" 201 369 "-" "PostmanRuntime/7.39.1" "-"

app-1 | Registered
nest_nginx | 172.20.0.1 - - [18/May/2026:18:42:20 +0000] "POST /auth/register HTTP/1.1" 201 369 "-" "PostmanRuntime/7.39.1" "-"
nest_mailhog | [APIv1] KEEPALIVE /api/v1/events
```